Java Grundlagen

Operatoren, Kontrollstrukturen und Methoden

Autor

Luis Staudt

30. November 2025

Inhaltsverzeichnis

Inhaltsverzeichnis 2

Operatoren in Java

Arithmetische Operatoren

Beschreibung:

Werden für mathematische Berechnungen verwendet.

Operator	Bedeutung	Beispiel	Ergebnis
+	Addition	5 + 3	8
-	Subtraktion	5 - 3	2
*	Multiplikation	5 * 3	15
/	Division	10 / 3	3 (bei int)
%	Modulo (Rest)	10 % 3	1

Beispiele:

```java
int a = 10;
int b = 3;
int summe = a + b;         // summe = 13
int differenz = a - b;     // differenz = 7
int produkt = a * b;       // produkt = 30
int quotient = a / b;      // quotient = 3 (Ganzzahldivision!)
int rest = a % b;          // rest = 1

// Gerade oder ungerade pruefen
int zahl = 7;
if (zahl % 2 == 0) {
    System.out.println("Gerade");
} else {
    System.out.println("Ungerade");
}
```

Wichtig bei Division:

```java
int x = 7 / 2;       // x = 3 (int-Division schneidet ab)
double y = 7.0 / 2; // y = 3.5 (mit Kommazahl)
```

Vergleichsoperatoren

Beschreibung:

Vergleichen zwei Werte und geben `true` oder `false` zurück.

Operator	Bedeutung	Beispiel	Ergebnis
==	Gleich	`5 == 5`	`true`
!=	Ungleich	`5 != 3`	`true`
>	Größer	`5 > 3`	`true`
<	Kleiner	`5 < 3`	`false`
>=	Größer oder gleich	`5 >= 5`	`true`
<=	Kleiner oder gleich	`5 <= 3`	`false`

Beispiele:

```java
int a = 10;
int b = 5;

if (a > b) {
    System.out.println("a ist groesser");
}

if (a == 10) {
    System.out.println("a ist 10");
}

if (b != 0) {
    int ergebnis = a / b;  // Division nur wenn b nicht 0 ist
}

// Array-Index pruefen
int[] zahlen = {1, 2, 3};
int index = 2;
if (index >= 0 && index < zahlen.length) {
    System.out.println(zahlen[index]);
}
```

Wichtig bei Strings:

```java
String text1 = "Hallo";
String text2 = "Hallo";

// FALSCH - vergleicht Objektreferenzen, nicht Inhalt!
if (text1 == text2) { }

// RICHTIG - vergleicht Inhalt
if (text1.equals(text2)) { }
```

Logische Operatoren

Beschreibung:

Verknüpfen mehrere Bedingungen.

Operator	Bedeutung	Beispiel	Ergebnis
&&	Und (AND)	`true && false`	`false`
||	Oder (OR)	`true || false`	`true`
!	Nicht (NOT)	`!true`	`false`

Beispiele:

```java
int alter = 20;
boolean hatFuehrerschein = true;

// UND: Beide Bedingungen muessen wahr sein
if (alter >= 18 && hatFuehrerschein) {
    System.out.println("Darf Auto fahren");
}

// ODER: Mindestens eine Bedingung muss wahr sein
int note = 1;
if (note == 1 || note == 2) {
    System.out.println("Gute Note!");
}

// NICHT: Kehrt Wahrheitswert um
boolean gefunden = false;
if (!gefunden) {
    System.out.println("Noch nicht gefunden");
}

// Array-Grenzen pruefen
int[] array = {1, 2, 3};
int i = 1;
if (i >= 0 && i < array.length) {
    System.out.println(array[i]);
}

// Zeichen ist Buchstabe oder Ziffer
char c = 'A';
if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) {
    System.out.println("Ist ein Buchstabe");
}
```

Kurzschluss-Auswertung:

```
// Bei && wird rechts nicht mehr ausgewertet, wenn links schon false ist
int[] zahlen = {1, 2, 3};
int index = 5;
if (index < zahlen.length && zahlen[index] > 0) {
    // Kein Fehler! zahlen[index] wird nicht ausgefuehrt, da index < zahlen
    .length bereits false ist
}
```

Zuweisungsoperatoren

Beschreibung:

Weisen einer Variable einen Wert zu oder ändern ihn.

Operator	Bedeutung	Beispiel	Entspricht
=	Zuweisung	a = 5	-
+=	Addition und Zuweisung	a += 3	a = a + 3
-=	Subtraktion und Zuweisung	a -= 3	a = a - 3
*=	Multiplikation und Zuweisung	a *= 3	a = a * 3
/=	Division und Zuweisung	a /= 3	a = a / 3
%=	Modulo und Zuweisung	a %= 3	a = a % 3

Beispiele:

```
int x = 10;
x += 5;   // x = 15 (entspricht: x = x + 5)
x -= 3;   // x = 12 (entspricht: x = x - 3)
x *= 2;   // x = 24 (entspricht: x = x * 2)
x /= 4;   // x = 6  (entspricht: x = x / 4)
x %= 4;   // x = 2  (entspricht: x = x % 4)

// Summe aufaddieren
int summe = 0;
int[] zahlen = {5, 10, 15};
for (int i = 0; i < zahlen.length; i++) {
    summe += zahlen[i];  // Kuerzer als: summe = summe + zahlen[i]
}

// Zeichen zaehlen
String text = "Hallo";
int anzahl = 0;
for (int i = 0; i < text.length(); i++) {
    if (text.charAt(i) == 'l') {
        anzahl++;  // Entspricht: anzahl = anzahl + 1
    }
}
```

Inkrement- und Dekrement-Operatoren

Beschreibung:

Erhöhen oder verringern einen Wert um 1.

Operator	Bedeutung	Beispiel	Ergebnis
++	Inkrement (erhöhen um 1)	`i++`	`i = i + 1`
-	Dekrement (verringern um 1)	`i-`	`i = i - 1`

Beispiele:

```java
int i = 5;
i++;        // i = 6 (entspricht: i = i + 1)
i--;        // i = 5 (entspricht: i = i - 1)

// In Schleifen
for (int j = 0; j < 5; j++) {  // j wird nach jedem Durchlauf um 1 erhoeht
    System.out.println(j);
}

// Countdown
int countdown = 10;
while (countdown > 0) {
    System.out.println(countdown);
    countdown--;
}

// Zaehler erhoehen
int anzahl = 0;
String text = "Hallo";
for (int k = 0; k < text.length(); k++) {
    if (text.charAt(k) == 'l') {
        anzahl++;
    }
}
```

Unterschied: Prä- vs. Post-Inkrement:

```java
int a = 5;
int b = a++;  // b = 5, dann a = 6 (Post-Inkrement: erst zuweisen, dann erhoehen)

int c = 5;
int d = ++c;  // c = 6, dann d = 6 (Prae-Inkrement: erst erhoehen, dann zuweisen)
```

String-Konkatenation

Beschreibung:

Der + Operator verbindet Strings.

Beispiele:

```
String vorname = "Anna";
String nachname = "Schmidt";
String vollname = vorname + " " + nachname;  // "Anna Schmidt"

// Mit Zahlen
int alter = 25;
String text = "Ich bin " + alter + " Jahre alt";  // "Ich bin 25 Jahre alt"

// In Schleifen
String wort = "Hallo";
String umgekehrt = "";
for (int i = wort.length() - 1; i >= 0; i--) {
    umgekehrt = umgekehrt + wort.charAt(i);
}
// umgekehrt = "ollaH"

// Array-Elemente verbinden
int[] zahlen = {1, 2, 3, 4, 5};
String ausgabe = "";
for (int i = 0; i < zahlen.length; i++) {
    ausgabe += zahlen[i];
    if (i < zahlen.length - 1) {
        ausgabe += ", ";
    }
}
// ausgabe = "1, 2, 3, 4, 5"
```

Operatorrangfolge (Priorität)

Beschreibung:

Operatoren werden in einer bestimmten Reihenfolge ausgewertet.

Von höchster zu niedrigster Priorität:

1. **Klammern**: ( )
2. **Inkrement/Dekrement**: ++, -
3. **Negation**: !
4. **Multiplikation, Division, Modulo**: *, /, %
5. **Addition, Subtraktion**: +, -
6. **Vergleichsoperatoren**: <, >, <=, >=
7. **Gleichheit**: ==, !=
8. **Logisches UND**: &&
9. **Logisches ODER**: ||
10. **Zuweisung**: =, +=, -=, etc.

Beispiele:

```
int ergebnis = 5 + 3 * 2;      // ergebnis = 11 (nicht 16!)
// * wird vor + ausgefuehrt: 5 + (3 * 2) = 5 + 6 = 11

int x = (5 + 3) * 2;            // x = 16
// Klammern aendern die Reihenfolge: (5 + 3) * 2 = 8 * 2 = 16

boolean b = 5 > 3 && 2 < 4;     // b = true
// Erst Vergleiche, dann &&: (5 > 3) && (2 < 4) = true && true = true

int y = 10 + 5 * 2 - 3;         // y = 17
// Reihenfolge: 5 * 2 = 10, dann 10 + 10 = 20, dann 20 - 3 = 17

// Bei Unsicherheit: Klammern verwenden!
int z = ((10 + 5) * 2) - 3;     // z = 27 (macht Absicht deutlich)
```

Worauf achten:

- Im Zweifelsfall immer Klammern setzen, um die Absicht klar zu machen!
- Multiplikation und Division haben höhere Priorität als Addition und Subtraktion
- Vergleichsoperatoren werden vor logischen Operatoren ausgewertet
- Klammern haben immer höchste Priorität

Kontrollstrukturen in Java

If-Bedingungen

Warum braucht man If-Bedingungen?

If-Bedingungen ermöglichen es, Code nur unter bestimmten Voraussetzungen auszuführen. Sie sind notwendig, wenn das Programm Entscheidungen treffen soll: "Wenn eine Bedingung erfüllt ist, dann führe diese Anweisungen aus.

Typische Anwendungsfälle:

- Prüfen, ob ein Wert einen bestimmten Zustand hat
- Vergleichen von Werten
- Reagieren auf Benutzereingaben
- Fehlerbehandlung (z.B. Index-Prüfung)

Einfache If-Bedingung

Syntax:

```
if (Bedingung) {
    Anweisung;
}
```

Beispiele:

```
// Zahl pruefen
int alter = 18;
if (alter >= 18) {
    System.out.println("Volljaehrig");
}

// Array-Index pruefen
int[] zahlen = {10, 20, 30};
int index = 1;
if (index >= 0 && index < zahlen.length) {
    System.out.println(zahlen[index]);  // Sicher zugreifen
}

// Zeichen pruefen
String text = "Hallo";
if (text.charAt(0) == 'H') {
    System.out.println("Beginnt mit H");
}

// String-Laenge pruefen
String wort = "Test";
if (wort.length() > 3) {
    System.out.println("Wort ist laenger als 3 Zeichen");
}

// Wert gefunden
boolean gefunden = false;
int[] liste = {5, 10, 15, 20};
int gesucht = 15;
for (int i = 0; i < liste.length; i++) {
    if (liste[i] == gesucht) {
        gefunden = true;
    }
}
```

If-Else-Bedingung

Syntax:

```
if (Bedingung) {
    Anweisung1;
} else {
    Anweisung2;
}
```

Beispiele:

```
// Gerade oder ungerade
int zahl = 7;
if (zahl % 2 == 0) {
    System.out.println("Gerade");
} else {
    System.out.println("Ungerade");
}

// Positiv oder negativ
int wert = -5;
if (wert >= 0) {
    System.out.println("Positiv oder Null");
} else {
    System.out.println("Negativ");
}

// Array leer oder gefuellt
int[] zahlen = {1, 2, 3};
if (zahlen.length > 0) {
    System.out.println("Array hat " + zahlen.length + " Elemente");
} else {
    System.out.println("Array ist leer");
}

// Zeichen vorhanden
String text = "Java";
if (text.length() > 0) {
    char erstes = text.charAt(0);
    System.out.println("Erstes Zeichen: " + erstes);
} else {
    System.out.println("String ist leer");
}
```

If-Else-If-Bedingung

Syntax:

```
if (Bedingung1) {
    Anweisung1;
} else if (Bedingung2) {
    Anweisung2;
} else {
    Anweisung3;
}
```

Beispiele:

```
// Notenbewertung
int punkte = 85;
if (punkte >= 90) {
    System.out.println("Note: Sehr gut");
} else if (punkte >= 80) {
    System.out.println("Note: Gut");
} else if (punkte >= 70) {
    System.out.println("Note: Befriedigend");
} else {
    System.out.println("Note: Nicht bestanden");
}

// Groessenvergleich
int a = 5;
int b = 10;
if (a > b) {
    System.out.println("a ist groesser");
} else if (a < b) {
    System.out.println("b ist groesser");
} else {
    System.out.println("a und b sind gleich");
}

// String-Laenge kategorisieren
String eingabe = "Hallo";
if (eingabe.length() == 0) {
    System.out.println("Leer");
} else if (eingabe.length() < 5) {
    System.out.println("Kurz");
} else if (eingabe.length() < 10) {
    System.out.println("Mittel");
} else {
    System.out.println("Lang");
}

```

```
36 // Index-Position bestimmen
37 int[] werte = {10, 20, 30, 40, 50};
38 int index = 2;
39 if (index < 0) {
40     System.out.println("Index zu klein");
41 } else if (index >= werte.length) {
42     System.out.println("Index zu gross");
43 } else {
44     System.out.println("Wert: " + werte[index]);
45 }
```

Verschachtelte If-Bedingungen

Syntax:

```
if (Bedingung1) {
    if (Bedingung2) {
        Anweisung;
    }
}
```

Beispiele:

```
// Doppelte Pruefung
String text = "Test";
if (text.length() > 0) {
    if (text.charAt(0) == 'T') {
        System.out.println("Beginnt mit T und ist nicht leer");
    }
}

// Array-Zugriff mit Pruefung
int[] zahlen = {5, 10, 15, 20};
int index = 2;
if (index >= 0 && index < zahlen.length) {
    if (zahlen[index] > 10) {
        System.out.println("Wert an Position " + index + " ist groesser als
    10");
    }
}

// Mehrere Bedingungen
int zahl = 24;
if (zahl > 0) {
    if (zahl % 2 == 0) {
        if (zahl < 100) {
            System.out.println("Positive gerade Zahl unter 100");
        }
    }
}
```

For-Schleifen

Warum braucht man For-Schleifen?

For-Schleifen ermöglichen es, Code mehrfach auszuführen. Sie sind besonders nützlich, wenn man:

- Alle Elemente eines Arrays durchlaufen möchte
- Alle Zeichen eines Strings prüfen will
- Eine Aufgabe eine bestimmte Anzahl wiederholen möchte
- Mit Indizes arbeiten muss (z.B. zum Ändern von Array-Werten)

Standard For-Schleife

Syntax:

```
for (Initialisierung; Bedingung; Inkrement) {
    Anweisung;
}
```

Wichtig: Alle drei Teile (Initialisierung, Bedingung, Inkrement) sind **optional** und können weggelassen werden!

Typisches Muster für Arrays:

```
for (int i = 0; i < array.length; i++) {
    // Verwende array[i]
}
```

Typisches Muster für Strings:

```
for (int i = 0; i < text.length(); i++) {
    // Verwende text.charAt(i)
}
```

Beispiele:

```
// Zahlen von 0 bis 4 ausgeben
for (int i = 0; i < 5; i++) {
    System.out.println(i);  // Gibt aus: 0, 1, 2, 3, 4
}

// Array durchlaufen
int[] zahlen = {10, 20, 30, 40, 50};
for (int i = 0; i < zahlen.length; i++) {
    System.out.println(zahlen[i]);
}

// String durchlaufen
String text = "Hallo";
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    System.out.println(c);
}

// Array-Werte aendern
int[] werte = {1, 2, 3, 4, 5};
for (int i = 0; i < werte.length; i++) {
    werte[i] = werte[i] * 2;  // Jeden Wert verdoppeln
}
// Ergebnis: {2, 4, 6, 8, 10}

// Array befuellen
```

```
27 int[] quadrate = new int[5];
28 for (int i = 0; i < quadrate.length; i++) {
29     quadrate[i] = i * i;
30 }
31 // Ergebnis: {0, 1, 4, 9, 16}
32
33 // Summe berechnen
34 int[] liste = {5, 10, 15, 20};
35 int summe = 0;
36 for (int i = 0; i < liste.length; i++) {
37     summe = summe + liste[i];
38 }
39 System.out.println("Summe: " + summe);  // Summe: 50
40
41 // Zeichen zaehlen
42 String wort = "Banane";
43 int anzahlA = 0;
44 for (int i = 0; i < wort.length(); i++) {
45     if (wort.charAt(i) == 'a') {
46         anzahlA++;
47     }
48 }
49 System.out.println("Anzahl 'a': " + anzahlA);  // Anzahl 'a': 3
50
51 // Rueckwaerts durchlaufen
52 int[] zahlen2 = {1, 2, 3, 4, 5};
53 for (int i = zahlen2.length - 1; i >= 0; i--) {
54     System.out.println(zahlen2[i]);  // Gibt aus: 5, 4, 3, 2, 1
55 }
56
57 // Nur gerade Indizes
58 int[] daten = {10, 20, 30, 40, 50, 60};
59 for (int i = 0; i < daten.length; i = i + 2) {
60     System.out.println(daten[i]);  // Gibt aus: 10, 30, 50
61 }
62
63 // Teilstring durchsuchen (von Position 2 bis Ende)
64 String satz = "Hallo Welt";
65 for (int i = 2; i < satz.length(); i++) {
66     System.out.println(satz.charAt(i));  // Gibt aus: l, l, o, , W, e, l, t
67 }
```

Optionale Teile der For-Schleife

Alle drei Teile können weggelassen werden!

Initialisierung weglassen

Wenn die Variable bereits vorher deklariert wurde:

```
// Beispiel 1: Zaehler bereits vorhanden
int i = 0;
for (; i < 5; i++) {
    System.out.println(i);
}

// Beispiel 2: Start-Index von aussen vorgegeben
int startIndex = 2;
for (; startIndex < 10; startIndex++) {
    System.out.println(startIndex);
}
```

Bedingung weglassen

Wenn die Bedingung immer wahr sein soll (Endlosschleife oder mit `break` beenden):

```
// Beispiel 1: Endlosschleife mit break
int[] zahlen = {10, 20, 30, 40, 50};
for (int i = 0; ; i++) {
    if (i >= zahlen.length) {
        break;  // Schleife manuell beenden
    }
    System.out.println(zahlen[i]);
}

// Beispiel 2: Suche mit vorzeitigem Abbruch
String text = "Hallo Welt";
for (int i = 0; ; i++) {
    if (i >= text.length()) {
        break;
    }
    if (text.charAt(i) == 'W') {
        System.out.println("Gefunden an Position: " + i);
        break;
    }
}
```

Inkrement weglassen

Wenn die Erhöhung manuell im Schleifenkörper erfolgt:

```
// Beispiel 1: Variable Schrittweite
int[] zahlen = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
for (int i = 0; i < zahlen.length; ) {
    System.out.println(zahlen[i]);
```

```
    if (zahlen[i] % 2 == 0) {
        i = i + 1;  // Nach geraden Zahlen: 1 Schritt
    } else {
        i = i + 2;  // Nach ungeraden Zahlen: 2 Schritte
    }
}

// Beispiel 2: Bedingte Erhoehung
String text = "aabbbcccc";
for (int i = 0; i < text.length(); ) {
    char c = text.charAt(i);
    System.out.println("Zeichen: " + c);
    // Ueberspringe alle gleichen Zeichen
    while (i < text.length() && text.charAt(i) == c) {
        i++;  // Erhoehung hier statt im for-Kopf
    }
}
```

Mehrere Teile weglassen

```
// Alle Teile weggelassen = Endlosschleife
// for (;;) {
//     System.out.println("Laeuft unendlich");
// }

// Nur Bedingung vorhanden
int i = 0;
for (; i < 5; ) {
    System.out.println(i);
    i++;
}
```

Verschachtelte For-Schleifen

Syntax:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        Anweisung;
    }
}
```

Beispiele:

```
// Multiplikationstabelle
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.println(i + " * " + j + " = " + (i * j));
    }
}

// Zwei Arrays vergleichen
int[] array1 = {1, 2, 3};
int[] array2 = {2, 3, 4};
for (int i = 0; i < array1.length; i++) {
    for (int j = 0; j < array2.length; j++) {
        if (array1[i] == array2[j]) {
            System.out.println("Gemeinsamer Wert: " + array1[i]);
        }
    }
}

// String-Vergleich (naiv)
String text1 = "abc";
String text2 = "bcd";
for (int i = 0; i < text1.length(); i++) {
    for (int j = 0; j < text2.length(); j++) {
        if (text1.charAt(i) == text2.charAt(j)) {
            System.out.println("Gemeinsames Zeichen: " + text1.charAt(i));
        }
    }
}

// Pattern ausgeben
for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}
// Ausgabe:
```

```
// *
// **
// ***
// ****
```

While-Schleifen

Warum braucht man While-Schleifen?

While-Schleifen verwendet man, wenn:

- Die Anzahl der Wiederholungen im Voraus nicht bekannt ist
- Die Schleife bis zum Eintreten einer bestimmten Bedingung laufen soll
- Man ßo lange wie nötig“wiederholen möchte
- Man nach einem bestimmten Element sucht und abbrechen will, wenn es gefunden wurde

Unterschied zur For-Schleife:

- For-Schleife: “Wiederhole genau n-malöder "Durchlaufe alle Elemente“
- While-Schleife: “Wiederhole solange diese Bedingung wahr ist“

While-Schleife

Syntax:

```
while (Bedingung) {
    Anweisung;
}
```

Beispiele:

```
// Zahlen von 0 bis 4 ausgeben
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}

// Element in Array suchen
int[] zahlen = {10, 20, 30, 40, 50};
int gesucht = 30;
int index = 0;
boolean gefunden = false;
while (index < zahlen.length && !gefunden) {
    if (zahlen[index] == gesucht) {
        gefunden = true;
        System.out.println("Gefunden an Position: " + index);
    }
    index++;
}

// Erste Position eines Zeichens finden
String text = "Hallo Welt";
char gesuchtesZeichen = 'W';
int position = 0;
while (position < text.length()) {
    if (text.charAt(position) == gesuchtesZeichen) {
        System.out.println("Gefunden an Position: " + position);
        break;  // Schleife beenden
    }
    position++;
}

// Solange Wert unter Grenze
int summe = 0;
int zahl = 1;
while (summe < 100) {
    summe = summe + zahl;
    zahl++;
}
```

```
40 System.out.println("Summe: " + summe);
41 System.out.println("Zahlen addiert: " + (zahl - 1));
42
43 // Array durchlaufen (alternative zu for)
44 int[] werte = {5, 10, 15, 20};
45 int idx = 0;
46 while (idx < werte.length) {
47     System.out.println(werte[idx]);
48     idx++;
49 }
50
51 // Bedingung wird nie erfuellt -> Schleife laeuft nicht
52 int x = 10;
53 while (x < 5) {
54     System.out.println("Wird nicht ausgegeben");
55     x++;
56 }
57
58 // Countdown
59 int countdown = 5;
60 while (countdown > 0) {
61     System.out.println(countdown);
62     countdown--;
63 }
64 System.out.println("Start!");
65
66 // Erstes Vorkommen eines Teilstrings finden
67 String satz = "Der Hund spielt im Garten";
68 String suche = "spielt";
69 int pos = 0;
70 boolean gefundenString = false;
71 while (pos <= satz.length() - suche.length() && !gefundenString) {
72     boolean match = true;
73     for (int j = 0; j < suche.length(); j++) {
74         if (satz.charAt(pos + j) != suche.charAt(j)) {
75             match = false;
76         }
77     }
78     if (match) {
79         gefundenString = true;
80         System.out.println("Gefunden an Position: " + pos);
81     }
82     pos++;
83 }
```

Vergleich: For vs. While

Wann For-Schleife?

```
// Wenn die Anzahl der Durchlaeufe bekannt ist
for (int i = 0; i < 10; i++) { }

// Beim Durchlaufen von Arrays
int[] zahlen = {1, 2, 3, 4, 5};
for (int i = 0; i < zahlen.length; i++) { }

// Beim Durchlaufen von Strings
String text = "Hallo";
for (int i = 0; i < text.length(); i++) { }

// Wenn man den Index braucht
for (int i = 0; i < array.length; i++) {
    array[i] = i * 2;  // Index wird zum Aendern gebraucht
}
```

Wann While-Schleife?

```
// Wenn die Anzahl der Durchlaeufe unbekannt ist
while (summe < 1000) {
    summe = summe + wert;
}

// Bei Suche mit vorzeitigem Abbruch
while (index < array.length && !gefunden) {
    if (array[index] == gesucht) {
        gefunden = true;
    }
    index++;
}

// Bedingungsabhaengige Wiederholung
while (bedingungErfuellt()) {
    macheEtwas();
}
```

Häufige Fehler

Endlosschleifen

```
// FALSCH - Endlosschleife!
int i = 0;
while (i < 5) {
    System.out.println(i);
    // i++ vergessen! Schleife laeuft unendlich
}

// RICHTIG
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;  // Zaehler erhoehen
}
```

Falsche Bedingungen

```
// FALSCH - ArrayIndexOutOfBoundsException!
int[] zahlen = {1, 2, 3};
for (int i = 0; i <= zahlen.length; i++) {  // <= ist falsch!
    System.out.println(zahlen[i]);
}

// RICHTIG
for (int i = 0; i < zahlen.length; i++) {  // < ist richtig
    System.out.println(zahlen[i]);
}
```

Semikolon-Fehler

```
// FALSCH - Semikolon beendet Schleife sofort
for (int i = 0; i < 5; i++);  // Semikolon hier ist falsch!
{
    System.out.println(i);  // Wird nur einmal ausgefuehrt, i nicht bekannt
}

// RICHTIG
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Off-by-One Fehler

```
String text = "Hallo";  // length = 5, Indizes: 0-4

// FALSCH
for (int i = 0; i <= text.length(); i++) {  // i = 5 ist zu viel!
    System.out.println(text.charAt(i));
}

// RICHTIG
for (int i = 0; i < text.length(); i++) {
    System.out.println(text.charAt(i));
}

// RICHTIG - Letztes Element direkt
char letztes = text.charAt(text.length() - 1);
```

Grundlegende String- und Array-Methoden in Java

String-Methoden

length() - Länge eines Strings

Was macht die Methode?

Gibt die Anzahl der Zeichen im String zurück.

Syntax:

```
int laenge = text.length();
```

Beispiele:

```
String name = "Anna";
int laenge = name.length();  // laenge = 4

String wort = "Hallo Welt";
int laenge2 = wort.length();  // laenge2 = 10 (Leerzeichen zaehlt mit!)

String leer = "";
int laenge3 = leer.length();  // laenge3 = 0

String satz = "Java";
for (int i = 0; i < satz.length(); i++) {
    // Schleife laeuft 4 mal (von 0 bis 3)
}
```

Worauf achten:

- Die Zählung der Zeichen beginnt bei 1 (erster Buchstabe = Position 0, aber length = 1)
- Leerzeichen und Sonderzeichen zählen mit
- Bei Strings: `.length()` **mit Klammern**
- Bei Arrays: `.length` **ohne Klammern**

charAt(int index) - Zeichen an Position

Was macht die Methode?

Gibt das Zeichen an der angegebenen Position zurück. Die Zählung beginnt bei 0!

Syntax:

```java
char zeichen = text.charAt(index);
```

Beispiele:

```java
String wort = "Hallo";
char erstes = wort.charAt(0);   // erstes = 'H'
char zweites = wort.charAt(1);  // zweites = 'a'
char letztes = wort.charAt(4);  // letztes = 'o'

// Alle Zeichen durchlaufen
String text = "Java";
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    System.out.println(c);  // Gibt aus: J, a, v, a
}

// Pruefung einzelner Zeichen
String eingabe = "A1B2";
if (eingabe.charAt(0) == 'A') {
    System.out.println("Beginnt mit A");
}

// Zeichen vergleichen
String code = "12345";
if (code.charAt(2) == '3') {
    System.out.println("An Position 2 steht eine 3");
}
```

Worauf achten:

- Index beginnt bei 0
- Index muss zwischen 0 und `text.length() - 1` liegen
- Bei ungültigem Index: `StringIndexOutOfBoundsException`
- Gibt ein `char` zurück, keinen `String`

Array-Operationen

array.length - Länge eines Arrays

Was macht es?

Gibt die Anzahl der Elemente im Array zurück. **Achtung:** Ist ein Attribut, keine Methode!

Syntax:

```
int laenge = array.length;  // OHNE Klammern!
```

Beispiele:

```
int[] zahlen = {10, 20, 30, 40, 50};
int anzahl = zahlen.length;  // anzahl = 5

String[] namen = new String[10];
int groesse = namen.length;  // groesse = 10 (auch wenn leer!)

// In Schleife verwenden
int[] werte = {1, 2, 3, 4};
for (int i = 0; i < werte.length; i++) {
    System.out.println(werte[i]);
}

// Letztes Element zugreifen
int[] liste = {100, 200, 300};
int letzter = liste[liste.length - 1];  // letzter = 300

// Leeres Array
int[] leer = new int[0];
int laenge = leer.length;  // laenge = 0
```

Worauf achten:

- `.length` **ohne Klammern** (bei Arrays)
- `.length()` **mit Klammern** (bei Strings)
- Gibt die Größe zurück, mit der das Array erstellt wurde
- Auch bei lleerenÄrrays (nur `null`-Werte) ist die Länge die Anzahl der Plätze

array[index] - Element lesen

Was macht es?

Greift auf das Element an der angegebenen Position im Array zu. Die Zählung beginnt bei 0!

Syntax:

```
Type element = array[index];
```

Beispiele:

```
int[] zahlen = {10, 20, 30, 40, 50};
int erstes = zahlen[0];   // erstes = 10
int zweites = zahlen[1];  // zweites = 20
int letztes = zahlen[4];  // letztes = 50

// Alle Elemente durchlaufen
String[] woerter = {"Hallo", "Welt", "Java"};
for (int i = 0; i < woerter.length; i++) {
    String wort = woerter[i];
    System.out.println(wort);
}

// Bestimmte Position pruefen
int[] noten = {1, 2, 3, 2, 1};
if (noten[2] == 3) {
    System.out.println("Dritte Note ist eine 3");
}

// Letztes Element
int[] liste = {5, 10, 15, 20};
int letzterWert = liste[liste.length - 1];  // letzterWert = 20

// Element in Berechnung verwenden
int[] preise = {100, 200, 150};
int summe = preise[0] + preise[1] + preise[2];  // summe = 450
```

Worauf achten:

- Index beginnt bei 0
- Index muss zwischen 0 und `array.length - 1` liegen
- Bei ungültigem Index: `ArrayIndexOutOfBoundsException`
- Letztes Element: `array[array.length - 1]`

array[index] = wert - Element schreiben

Was macht es?

Weist dem Element an der angegebenen Position einen neuen Wert zu.

Syntax:

```
array[index] = neuerWert;
```

Beispiele:

```
int[] zahlen = new int[5];  // Array mit 5 Plaetzen, alle 0
zahlen[0] = 10;  // Erstes Element auf 10 setzen
zahlen[1] = 20;  // Zweites Element auf 20 setzen
zahlen[4] = 50;  // Letztes Element auf 50 setzen
// Ergebnis: {10, 20, 0, 0, 50}

// Werte aendern
String[] namen = {"Anna", "Bob", "Clara"};
namen[1] = "Max";  // "Bob" wird zu "Max"
// Ergebnis: {"Anna", "Max", "Clara"}

// In Schleife befuellen
int[] quadrate = new int[5];
for (int i = 0; i < quadrate.length; i++) {
    quadrate[i] = i * i;
}
// Ergebnis: {0, 1, 4, 9, 16}

// Wert erhoehen
int[] punkte = {10, 20, 30};
punkte[0] = punkte[0] + 5;  // Erstes Element um 5 erhoehen
// Ergebnis: {15, 20, 30}

// Array "leeren" (auf Standard-Wert setzen)
int[] werte = {1, 2, 3, 4, 5};
for (int i = 0; i < werte.length; i++) {
    werte[i] = 0;
}
// Ergebnis: {0, 0, 0, 0, 0}

// Zwei Elemente tauschen
int[] liste = {10, 20, 30};
int temp = liste[0];
liste[0] = liste[2];
liste[2] = temp;
// Ergebnis: {30, 20, 10}
```

Worauf achten:

- Index muss zwischen 0 und `array.length - 1` liegen
- Bei ungültigem Index: `ArrayIndexOutOfBoundsException`
- Der Wert muss zum Datentyp des Arrays passen
- Überschreibt den vorherigen Wert an dieser Position

Häufige Fehler und Verwechslungen

String vs. Array - length

```
// FALSCH
String text = "Hallo";
int laenge = text.length;  // Fehler: Strings brauchen ()

int[] zahlen = {1, 2, 3};
int anzahl = zahlen.length();  // Fehler: Arrays haben KEINE ()

// RICHTIG
int laenge = text.length();    // String MIT Klammern
int anzahl = zahlen.length;    // Array OHNE Klammern
```

Index-Grenzen

```
String wort = "Test";  // length = 4, Indizes: 0, 1, 2, 3
char c = wort.charAt(4);  // Fehler! Index 4 existiert nicht

int[] zahlen = {10, 20, 30};  // length = 3, Indizes: 0, 1, 2
int x = zahlen[3];  // Fehler! Index 3 existiert nicht

// RICHTIG
char letztes = wort.charAt(wort.length() - 1);  // Index 3
int letzter = zahlen[zahlen.length - 1];  // Index 2
```

char vs. String

```
String text = "A";
char buchstabe = 'A';

// FALSCH
if (text.charAt(0) == "A") { }  // Vergleicht char mit String

// RICHTIG
if (text.charAt(0) == 'A') { }  // char mit char vergleichen
```